

TECHCONF

¿CÓMO SE HIZO?



UNIVERSIDAD DE GRANADA

TECNOLOGÍAS WEB: PROYECTO WEB

- Equipo de Desarrollo -
Javier Rivera Delgado
Diego Romero Fuentes
Guillermo Martín Sánchez

1. Introducción y Objetivos del Proyecto.....	2
1.1. Propósito del sistema (Plataforma TechConf).....	2
1.2. Requisitos funcionales y no funcionales.....	2
2. Arquitectura del Sistema y Pila Tecnológica.....	2
2.1. Elección del Framework (Laravel) y Patrón MVC.....	2
2.2. Tecnologías de Frontend (HTML5 Semántico y CSS Puro).....	3
3. Modelado de Datos.....	3
3.1. Entidades principales (Usuarios, Talleres, Inscripciones, Materiales).....	3
3.2. Relaciones.....	4
4. Implementación del Backend y Lógica de Negocio.....	5
4.1. Sistema de Autenticación y Autorización (Roles: Asistente vs. Organizador).....	5
4.2. Gestión del Perfil de Usuario (CRUD completo y seguridad de contraseñas).....	5
4.3. Módulo de Gestión de Eventos (Talleres y control de aforos).....	6
4.4. Repositorio de Materiales Docentes.....	6
5. Interfaz de Usuario (UI) y Experiencia (UX).....	6
5.1. Diseño del Layout Base y Panel Contextual (Widgets).....	6
5.2. Componentización de Vistas (Alertas centralizadas).....	7
5.3. Criterios de Accesibilidad y Usabilidad.....	7
6. Seguridad y Buenas Prácticas.....	7
6.1. Protección contra vulnerabilidades (CSRF, Inyección SQL).....	7
6.2. Encriptación y protección de datos sensibles.....	7

1. Introducción y Objetivos del Proyecto

1.1. Propósito del sistema (Plataforma TechConf)

El presente proyecto detalla el diseño e implementación de "TechConf", una plataforma web integral destinada a la gestión de conferencias y talleres tecnológicos. El objetivo principal ha sido desarrollar un sistema robusto que permita automatizar el ciclo de vida completo de un evento formativo, sirviendo de nexo entre los asistentes y el equipo organizador. La plataforma busca eliminar la fricción en los procesos de inscripción, ofreciendo un panel centralizado donde los usuarios pueden gestionar su agenda y acceder a los recursos didácticos de forma intuitiva.

1.2. Requisitos funcionales y no funcionales

Para garantizar la viabilidad del proyecto, se establecieron una serie de requisitos fundamentales que guían tanto el desarrollo técnico como la experiencia de usuario. A nivel funcional, el sistema debía soportar el registro seguro de usuarios mediante protocolos de validación rigurosos, la discriminación de roles (Asistente y Organizador) para segmentar las capacidades de administración, la matriculación en talleres con un motor de control de aforo en tiempo real que evite el sobrecupo de forma automática, y una gestión documental avanzada que permita la subida y descarga de materiales adjuntos asociados a cada evento formativo.

A nivel no funcional, se ha priorizado la mantenibilidad del código mediante una separación estricta de responsabilidades bajo el patrón MVC, asegurando que la lógica de negocio permanezca aislada de la presentación. Asimismo, la seguridad de las transacciones web se ha reforzado mediante la protección contra ataques comunes como CSRF e inyección SQL, mientras que la accesibilidad de la interfaz se ha trabajado de forma nativa, prescindiendo deliberadamente de librerías de estilos de terceros para garantizar un control absoluto sobre el código visual y optimizar los tiempos de carga del sistema.

2. Arquitectura del Sistema y Pila Tecnológica

2.1. Elección del Framework (Laravel) y Patrón MVC

El desarrollo del *backend* se ha cimentado sobre Laravel, un *framework* de PHP de código abierto seleccionado por su madurez, expresividad y su ecosistema de herramientas integradas. La arquitectura del software orbita en torno al patrón MVC (Modelo-Vista-Controlador), lo que ha permitido encapsular la lógica de acceso a datos a través del ORM Eloquent (Modelos), gestionar el flujo de las peticiones HTTP y la lógica de negocio a través de los Controladores, y presentar la información al usuario final mediante el motor de plantillas Blade (Vistas).

Esta elección tecnológica facilita una estructura de archivos estandarizada, promoviendo un código limpio y escalable. Gracias a las migraciones de Laravel, se garantiza una gestión de

la base de datos versionada y coherente, mientras que el sistema de enrutamiento permite definir puntos de entrada claros para cada funcionalidad del sistema TechConf.

2.2. Tecnologías de Frontend (HTML5 Semántico y CSS Puro)

Para la estructuración del contenido web se ha empleado HTML5 semántico. El uso de etiquetas estructurales específicas (como <header>, <nav>, <main>, <aside> para los paneles laterales y <footer>) mejora la legibilidad y el mantenimiento del código y es fundamental para la accesibilidad web y la correcta indexación de los motores de búsqueda. Gracias a esto, la información del evento, la agenda y los formularios quedan jerarquizados de forma lógica y estandarizada.

En cuanto a la capa de presentación, hemos optado por el desarrollo íntegro en CSS puro, descartando la implementación de frameworks externos. Esto lo decidimos así por la necesidad de mantener el control máximo sobre el diseño visual. Para configurar la disposición de los elementos en pantalla de forma fluida y adaptable, se ha hecho uso de tecnologías como Flexbox, combinadas con Media Queries. Esto ha permitido desarrollar una arquitectura de estilos Responsive, garantizando una correcta visualización y reestructuración de la interfaz en cualquier resolución de pantalla, desde dispositivos móviles hasta monitores de escritorio.

3. Modelado de Datos

3.1. Entidades principales (Usuarios, Talleres, Inscripciones, Materiales)

En esta parte se definió la estructura de datos necesaria para que la plataforma TechConf funcionara de forma dinámica. Para ello se crearon migraciones de Laravel para las tablas eventos, talleres, inscripciones, materiales y propuestas_eventos, además de ampliar la tabla users con el campo rol.

La tabla **users** la diseñamos para que contenga los datos de acceso y el campo rol, utilizado para diferenciar usuarios de tipo asistente/organizador.

La tabla **eventos** almacena el evento principal TechConf, mientras que la tabla talleres contiene las sesiones concretas de la agenda: título, descripción, ponente, fecha, hora de inicio, hora de fin, aula y aforo. Esta última columna es completamente necesaria para calcular las plazas disponibles.

La tabla **inscripciones** relaciona usuarios con talleres mediante user_id y taller_id. También incorpora el campo añadido “*asistio*”, que permite al organizador marcar si un usuario acudió o no a un taller. Para evitar duplicados, la migración incluye una restricción “*unique*” a la pareja user_id y taller_id, para no permitir que un mismo asistente se inscriba a dos talleres.

La tabla **materiales** guarda los documentos de apoyo asociados a un taller. En lugar de guardar el archivo completo en la base de datos, se almacena la ruta relativa del fichero subido al servidor.

La última tabla es **propuestas_eventos**, que se añadió para permitir que un usuario autenticado proponga nuevos talleres, quedando inicialmente en estado pendiente hasta que el organizador decida aceptarlos o rechazarlos.

3.2. Relaciones

Relaciones principales implementadas:

Evento - Taller

- Relación de uno a muchos.
- Un evento puede tener muchos talleres.
- Cada taller pertenece a un único evento.
- En la base de datos se implementó mediante la columna `evento_id` en la tabla `talleres`.
- En los modelos se representó con:
 - `Evento::talleres()`
 - `Taller::evento()`

Taller - Inscripción

- Relación de uno a muchos.
- Un taller puede tener muchas inscripciones.
- Cada inscripción pertenece a un único taller.
- Esta relación permite contar los asistentes inscritos en cada taller.

Usuario - Inscripción

- Relación de uno a muchos.
- Un usuario puede tener varias inscripciones.
- Cada inscripción pertenece a un único usuario.
- En los modelos se implementó con:
 - `User::inscripciones()`
 - `Inscripcion::user()`

Taller - Material

- Relación de uno a muchos.
- Un taller puede tener varios materiales de apoyo asociados.
- Cada material pertenece a un único taller.
- Esto permite mostrar en el detalle de cada taller los documentos disponibles para descargar.
- En los modelos se implementó con:

- Taller::materiales()
- Material::taller()

Usuario - PropuestaEvento

- Relación de uno a muchos.
- Un usuario logueado puede enviar varias propuestas de eventos o talleres.
- Cada propuesta pertenece al usuario que la ha enviado.
- Esta relación se implementó mediante `user_id` en la tabla `propuestas_eventos`.
- En los modelos se representó con:
 - User::propuestasEventos()
 - PropuestaEvento::user()

4. Implementación del Backend y Lógica de Negocio

4.1. Sistema de Autenticación y Autorización (Roles: Asistente vs. Organizador)

La seguridad del acceso se ha gestionado mediante el *AuthController*. Para el registro, se implementó una validación estricta que exige confirmación de contraseña y garantiza la unicidad del correo electrónico. Las contraseñas se protegen utilizando el algoritmo de hashing `bcrypt` (*Hash::make*) antes de su persistencia.

El control de acceso (*Autorización*) se apoya en el sistema de Middlewares de Laravel. Se han protegido las rutas sensibles verificando el rol del usuario autenticado; de este modo, un usuario con rol de asistente es redirigido a su área personal, impidiendo cualquier acceso no autorizado a las rutas del panel de administración exclusivas del organizador.

4.2. Gestión del Perfil de Usuario (CRUD completo y seguridad de contraseñas)

Se ha desarrollado un módulo integral (*ProfileController*) que otorga al asistente control total sobre sus datos, cumpliendo con los estándares actuales de usabilidad y privacidad:

- **Actualización de datos:** Permite modificar el nombre y correo electrónico, implementando una regla de validación que ignora el ID del usuario actual para evitar conflictos de "correo duplicado" al guardar.
- **Cambio de credenciales:** Requiere la validación de la contraseña actual (*current_password*) como medida de seguridad antes de procesar y encriptar la nueva clave.

- **Eliminación de cuenta:** Para prevenir borrados accidentales o malintencionados (en caso de sesión expuesta), se exige una confirmación mediante contraseña antes de proceder con el borrado definitivo del registro y la invalidación de la sesión.

4.3. Módulo de Gestión de Eventos (Talleres y control de aforos)

El núcleo operativo recae sobre el *TallerController*. Para la visualización de la agenda, se ha programado un sistema dinámico de filtrado que permite consultar los eventos mediante parámetros en la URL (peticiones GET).

A través del uso de funciones anónimas en las consultas del ORM (*where(function(\$q...)*), se garantiza que los filtros de búsqueda por texto (título o ponente) no interfieran con los filtros estructurales (como la visualización por días específicos). Adicionalmente, el controlador inyecta variables estadísticas calculadas al vuelo (total de talleres e inscritos) para nutrir los widgets del panel lateral de la interfaz.

4.4. Repositorio de Materiales Docentes

La gestión de archivos se ha abordado mediante el *MaterialController*. Los organizadores disponen de una interfaz para asociar documentos a talleres específicos. El backend procesa las peticiones de tipo *multipart/form-data*, almacenando los archivos de forma segura en el sistema de ficheros del servidor (*storage/app/public*) y guardando únicamente la ruta relativa en la base de datos. Se ha completado el ciclo de vida de este recurso implementando la operación de borrado (*destroy*), que localiza el material mediante Model Binding y lo elimina del inventario.

5. Interfaz de Usuario (UI) y Experiencia (UX)

5.1. Diseño del Layout Base y Panel Contextual (Widgets)

Para la estructura visual del proyecto, hemos diseñado una base adaptable, asegurando que sea perfectamente visualizado en dispositivos móviles, tablets... La maquetación se ha construido en base a una estructura que incorpora un menú de navegación superior, una amplia zona central para la carga dinámica de contenido y un menú lateral destinado a los widgets de la plataforma. Para mantener la coherencia entre páginas, se ha implementado un pie de página compartido por todos los documentos, el cual alberga los enlaces institucionales, la página de contacto con la información del equipo de desarrollo y el acceso directo al presente informe técnico de la plataforma.

5.2. Componentización de Vistas (Alertas centralizadas)

Haciendo uso del motor de plantillas Blade, hemos establecido un sistema de vistas modulares que aseguran que todos los documentos heredan la misma cabecera, estructura principal y pie de página. Para la vista pública de los asistentes, hemos diseñado la interfaz de la agenda general, la cual expone los talleres disponibles y permite la descarga del programa, junto con las vistas detalladas de cada evento y la página informativa de contacto. A nivel de presentación, toda la lógica de estilos se ha organizado en ficheros CSS externos y totalmente separados de la estructura HTML, facilitando su mantenimiento y escalabilidad.

5.3. Criterios de Accesibilidad y Usabilidad

De acuerdo a los requisitos no funcionales del proyecto, la capa de presentación se ha implementado utilizando HTML5 Semántico y CSS Puro. Tal y como se ha mencionado anteriormente, hemos decidido prescindir de librerías de estilos de terceros para tener control absoluto sobre el código visual de la plataforma y optimizar los tiempos de carga. Así hemos conseguido que la interfaz sea ligera, accesible e intuitiva, y se elimina la dificultad del usuario a la hora de consultar la información pública y navegar por la agenda del evento.

6. Seguridad y Buenas Prácticas

6.1. Protección contra vulnerabilidades (CSRF, Inyección SQL)

En la parte que corresponde a la base de datos, se utilizó la directiva “@csrf” de Blade, la cual genera un token de seguridad que Laravel comprueba al recibir peticiones (como POST, PUT, DELETE...). Fue aplicada en los formularios de creación y edición de talleres, subida de materiales, asistencia...

6.2. Encriptación y protección de datos sensibles

Hemos protegido las rutas administrativas mediante la comprobación del rol del usuario antes de ejecutar acciones sensibles. No es suficiente con que el usuario haya iniciado sesión; para acceder al panel, gestionar talleres, subir materiales o revisar propuestas debe tener rol organizador.

Esta comprobación se la implementamos en los controladores mediante la definición de la función **comprobarOrganizador()**. Si el usuario no está autenticado o su rol no es organizador, se ejecuta abort (error 403), impidiendo el acceso incluso si intenta escribir manualmente la URL en el navegador.

De esta forma, las principales operaciones críticas de administración las dejamos restringidas al organizador: creación y borrado de talleres, consulta de inscritos, modificación de asistencia, gestión de materiales y aceptación o rechazo de propuestas.